

Python Topic 7 — Dictionaries

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert • Python's most-used data structure

Layer 1 — New to Python

A **dictionary** stores data as **key** → **value** pairs — Python's version of a JavaScript object (or **Map**). Curly braces **{}**, with **key: value** inside.

```
person = {  
    "name": "Akshay",  
    "age": 26,  
    "city": "Pune"  
}
```

Look things up by **key** (not by position like a list):

```
person["name"]    # "Akshay"  
person["age"]     # 26
```

Add or change values:

```
person["email"] = "akshay@example.com"    # add a new key  
person["age"] = 27                        # change an existing value
```

 **Mental model:** like a real dictionary — look up a **word** (key) to get its **definition** (value).

Layer 2 — Intermediate (real code + how it works)

JS object vs Python dict

```
// JS  
const person = { name: "Akshay", age: 26 };  
person.name;           // dot access  
person["name"];        // bracket access  
  
# Python — ONLY bracket access, no dot  
person = {"name": "Akshay", "age": 26}
```

```
person["name"]          # "Akshay"
# person.name ❌ doesn't work in Python!
```

i Big difference: Python keys are usually **strings in quotes**, accessed **only with brackets**, never dot notation.

Safe access with `.get()`

```
person["phone"]          # ❌ KeyError – crashes if key missing
person.get("phone")       # None – returns None instead of crashing
person.get("phone", "N/A") # "N/A" – provide a default
```

Like JS `obj.phone ?? "N/A"` — use `.get()` whenever a key might not exist.

Essential methods

```
person.keys()            # dict_keys(['name', 'age']) → all keys
person.values()          # dict_values(['Akshay', 26]) → all values
person.items()           # dict_items([('name', 'Akshay'), ('age', 26)]) → pairs
person.pop("age")         # removes 'age' and returns its value
"name" in person          # True → checks KEYS (not values)
len(person)              # number of key-value pairs
person.update({"age": 27, "city": "Pune"}) # merge multiple updates
```

Looping — the three patterns you'll use daily

```
for key in person:                # loops over KEYS by default
    print(key)

for key, value in person.items():  # loops over both → most common
    print(f"{key}: {value}")

for value in person.values():      # just the values
    print(value)
```

Nested dicts (like JSON from an API)

```
data = {
    "user": {
        "name": "Akshay",
        "skills": ["React", "Python"]
    }
}
```

```
}  
data["user"]["skills"][0]    # "React" → chain the brackets
```

Exactly how you'll navigate JSON responses — dicts and lists nested together.

Layer 3 — Expert (internals & gotchas)

1. Dicts preserve insertion order (since Python 3.7)

```
d = {"z": 1, "a": 2, "m": 3}  
list(d.keys())    # ['z', 'a', 'm'] → insertion order, guaranteed
```

Older Python (<3.7) didn't guarantee this. Now it's part of the language spec — same as JS objects.

2. Keys must be hashable (immutable) — connects to Topic 6

```
{"name": "ok"}           # ✓ string key  
{(1, 2): "point"}       # ✓ tuple key  
[[1, 2]: "bad"]         # ✗ TypeError – list can't be a key
```

Values can be anything; only **keys** must be immutable/hashable.

Can be a key?	Type	Example	Why
✓ Yes	str	"name"	immutable
✓ Yes	int	42	immutable
✓ Yes	float	3.14	immutable
✓ Yes	bool	True	immutable
✓ Yes	tuple (of immutables)	(1, 2)	immutable & hashable
✓ Yes	frozenset	frozenset([1,2])	immutable set
✓ Yes	None	None	immutable
✗ No	list	[1, 2]	mutable → unhashable
✗ No	dict	{"a": 1}	mutable → unhashable
✗ No	set	{1, 2}	mutable → unhashable
⚠ Depends	tuple with a mutable inside	(1, [2, 3])	unhashable — a list inside breaks it

⚠ **Subtle row:** a tuple is *usually* fine as a key, but if it contains a mutable object (like a list), it becomes unhashable and fails — `hash((1, [2,3]))` raises `TypeError`.

3. Dict comprehensions

```
squares = {n: n**2 for n in range(5)}    # {0:0, 1:1, 2:4, 3:9, 4:16}

# swap keys and values
person = {"name": "Akshay", "age": 26}
flipped = {v: k for k, v in person.items()}
# flipped → {"Akshay": "name", 26: "age"}

# filter – keep only entries where value >= 18
ages = {"Akshay": 26, "Riya": 15, "Sam": 20}
adults = {k: v for k, v in ages.items() if v >= 18}
# adults → {"Akshay": 26, "Sam": 20}
```

4. Merging dicts — the modern `|` operator (Python 3.9+)

```
a = {"x": 1, "y": 2}
b = {"y": 99, "z": 3}
a | b      # {"x":1, "y":99, "z":3} → b wins on conflicts (like JS {...a, ...b})
{**a, **b} # older syntax, same result
```

5. `setdefault` / `defaultdict` — grouping & counting

```
# A real list of words – e.g. tags extracted from a document
words = ["python", "ai", "python", "rag", "python", "ai"]

# Count occurrences – the classic pattern
from collections import defaultdict
counts = defaultdict(int)
for w in words:
    counts[w] += 1      # no KeyError – missing keys default to 0
# counts → {'python': 3, 'ai': 2, 'rag': 1}

# Or use Counter (even cleaner – one line)
from collections import Counter
counts = Counter(words) # Counter({'python': 3, 'ai': 2, 'rag': 1})
```

✓ **Time-savers:** `Counter` and `defaultdict` are ones you'll reach for constantly in data processing.

6. The shared-reference trap (dicts are mutable)

```

a = {"x": 1}
b = a          # NOT a copy – same dict
b["x"] = 99
print(a)       # {"x": 99} → a changed too!

b = a.copy()   # shallow copy → JS {...a}
import copy
b = copy.deepcopy(a)  # deep copy for nested dicts

```

7. **dict** ↔ JSON — you'll do this constantly for APIs

```

import json
json_string = json.dumps(person)    # dict → JSON string (for sending)
data = json.loads(json_string)      # JSON string → dict (for receiving)

```

i Naming: **dumps** = "dump string", **loads** = "load string". Bread and butter of API work.

Interview Q&A Cards

Q: **d["key"]** vs **d.get("key")** ?

A: **d["key"]** raises **KeyError** if the key is missing. **d.get("key")** returns **None** (or a supplied default) instead of crashing. Use **.get()** when a key may not exist.

Q: Are Python dicts ordered?

A: Yes — insertion order is guaranteed since Python 3.7 (part of the spec). Earlier versions didn't guarantee it.

Q: What can be a dict key?

A: Any hashable (immutable) object — strings, numbers, tuples. Lists and dicts can't be keys. Values can be anything.

Q: How do you count occurrences of items?

A: **collections.Counter(iterable)**, or a **defaultdict(int)** incrementing per key. Both avoid the **KeyError** of a plain dict.

Q: Convert a dict to/from JSON?

A: **json.dumps(d)** → JSON string (sending); **json.loads(s)** → dict (receiving). "dump string" / "load string".

🔗 **Memory hook:** Dict = JS object, but brackets only, no dot. `.get()` won't crash. `dumps` sends, `loads` receives.

✅ **One-liner for interviews:** "Dicts are hashable-key → value maps with O(1) lookup, insertion-ordered since 3.7. Keys must be immutable. Use `.get()` for safe access, dict comprehensions to transform, `|` to merge, and `Counter` / `defaultdict` for grouping. `json.dumps` / `loads` convert to and from JSON."